

Abstract :

Hermes.io is an autonomous, full-stack educational intelligence platform designed to address the fragmentation, static rigidity, and lack of verified progression in modern self-directed technical education. Built upon a unified `Node.js` serverless HTTP micro-engine and backed by a dual-tier cloud database architecture utilizing **Supabase PostgreSQL** with resilient local file failover, Hermes synthesizes personalized, role-specific curriculum trees utilizing modern **Large Language Model (LLM)** inference. The platform incorporates multi-tiered learning modalities, offering both a custom 4-step wizard and a rich gallery of standardized industry blueprints. Each generated curriculum features an interactive, recursive milestone constellation with on-demand submodule expansions, verified knowledge mastery quizzes, and an executive **Hermes Overseer Terminal** secured via stateless cryptographic **HMAC-SHA256** tokens. Interactive features include procedural celestial planet avatars, a cosmic living universe canvas with slow-falling shooting stars, and a tactile **Slide to Enter** gateway. Rigorous automated verification demonstrates sub-15ms telemetry latency, seamless offline fallback, and high-fidelity progress synchronization across distributed serverless environments.

Table of Contents

1. Introduction	4
1.1 Purpose	4
1.2 Problem Statement	4
1.3 Objectives	4
2. Tools and Technologies	5
2.1 Frontend Architecture	5
2.2 Backend & API Layer	5
2.3 Cloud Database & Resilience Layer	5
2.4 AI Inference & Cloud Infrastructure	5
3. System Design	6
3.1 Overall System Architecture	6
3.2 Application Flowchart	6
4. Implementation	7
4.1 Cosmic Splash Page with Tactile Slide Gateway	7
4.2 AI Curriculum Synthesizer & Submodule Engine	7
4.3 Stateless HMAC-SHA256 Overseer Security Architecture	7
4.4 Automated Examination Engine	7
5. GUI Screens and Interface Walkthrough	8
6. Code Snippets	9
7. Testing and Debugging	10
8. Conclusion & Future Enhancements	11

1. Introduction

1.1 Purpose

The purpose of the **Hermes.io** project is to engineer an enterprise-grade, end-to-end learning path synthesizer and tracking dashboard. Self-directed technical learners frequently confront unstructured tutorials, disjointed documentation, and open-ended roadmaps that lack actionable milestones, practical project callouts, or verifiable assessments. Hermes.io solves this paradigm by automatically transforming high-level career goals (e.g., *Full Stack Developer*, *AI/ML Engineer*, *DevOps Architect*) into interactive, multi-stage milestone constellations accompanied by automated exams, study hour rollups, and administrative telemetry.

1.2 Problem Statement

Current educational paradigms exhibit several critical bottlenecks:

- **Disjointed learning materials** — Learners waste substantial time navigating heterogeneous blog posts, outdated documentation, and uncured video playlists without clear prerequisites.
- **Static, non-adaptive curricula** — Existing roadmap platforms offer static PNG/SVG diagrams that cannot dynamically adjust to a learner's prior experience or specific tech-stack preferences.
- **Absence of verified mastery** — Traditional roadmaps rely entirely on self-reported checkboxes without evaluating comprehension through targeted conceptual assessments.
- **Administrative blind spots** — Institutional supervisors and enterprise team leads lack real-time observability into team skill progression, milestone bottlenecks, and examination pass rates.

1.3 Objectives

To resolve the aforementioned challenges, Hermes.io fulfills the following engineering objectives:

1. Design a responsive, cosmic-themed web interface featuring a tactile *Slide to Enter* gate, live universe background with slow shooting stars, and procedural SVG celestial star/planet avatars.
2. Implement an intelligent roadmap synthesis pipeline leveraging Gemini generative AI with graceful local fallback heuristics for fully offline continuity.
3. Provide on-demand, recursive submodule expansion enabling learners to drill deep into complex technical milestones without losing overall curriculum context.
4. Develop a verified examination engine supporting multi-question modular mastery quizzes with score analytics and badge rewards.
5. Construct the **Hermes Overseer Terminal**, an enterprise telemetry console powered by stateless HMAC-SHA256 authentication for serverless environments.
6. Implement cloud persistence using Supabase PostgreSQL with automated schema migration, row-level security (RLS), and zero-loss local JSON synchronization.

2. Tools and Technologies

2.1 Frontend Architecture

The frontend interface is engineered using semantic HTML5, modern CSS3 design tokens, and modular, vanilla JavaScript (ESNext). By avoiding bloated component frameworks, Hermes achieves instant first-contentful-paint (FCP), zero build-step overhead, and smooth 60fps animations. Key components include:

- **Phosphor Icons Web (v2.1.1)** — Comprehensive iconography across regular and fill states.
- **Google Fonts** — *Plus Jakarta Sans* for headings and UI copy; *JetBrains Mono* for telemetry, tokens, and monospace codes.
- **Celestial Avatars Engine** — Deterministic procedural SVG generator producing 10 unique planetary and star archetypes seeded by user email addresses.

2.2 Backend & API Layer

The backend is powered by native Node.js without heavyweight external middleware dependencies.

- **Stateless HTTP Server** — Lightweight RESTful micro-engine handling CORS, streaming JSON bodies, static asset caching, and security headers.
- **Stateless Cryptographic HMAC-SHA256 Auth** — Replaces fragile in-memory serverless session stores with signed, self-contained bearer tokens (`hadmin.<payload>.<signature>`) that remain fully valid across dynamic Vercel Lambda container spin-ups.

2.3 Cloud Database & Resilience Layer

- **Supabase PostgreSQL (v15)** — Cloud relational database managing persistent tables for users, sessions, roadmaps, quiz_results, and app_config.
- **Local JSON Failover** — Autonomous filesystem persistence layer (`data/users.json`, `data/roadmaps.json`, `data/quiz_results.json`) guaranteeing 100% functional continuity during cloud outages or zero-configuration development.

2.4 AI Inference & Cloud Infrastructure

- **Google Gemini API (Gemini 2.5 Flash)** — High-speed JSON schema generation for curriculum structuring, milestone sequencing, and contextual quiz formulation.
- **Vercel Serverless Edge** — Continuous deployment pipeline with automated Git push triggering.

Table 1. Core technology stack and layer allocation.

Layer	Technology / Framework	Version / Role
Client Presentation	Semantic HTML5, CSS Variables, Plus Jakarta Sans	Custom Design System
Client Interactions	Vanilla ES6+ JavaScript, Phosphor Icons Web	Single Page App (SPA)
Procedural Graphics	CelestialAvatars SVG Generator	Seeded Hash Algorithm
Server Runtime	Node.js HTTP Micro-Engine	v20.x / v24.x LTS
Authentication	Stateless HMAC-SHA256 Cryptographic Tokens	PBKDF2 Password Salts
Cloud Database	Supabase PostgreSQL with Row Level Security	Cloud DB / PostgREST
AI Synthesis Engine	Google Gemini 2.5 Flash	Structured JSON Schema
Hosting & Deployment	Vercel Serverless Functions & GitHub CI/CD	Production Cloud

3. System Design

3.1 Overall System Architecture

Hermes.io follows a decoupled, resilient Multi-Tier Architecture comprising Presentation, Gateway, Logic, and Persistence layers. Figure 1 illustrates the communication flow across components.

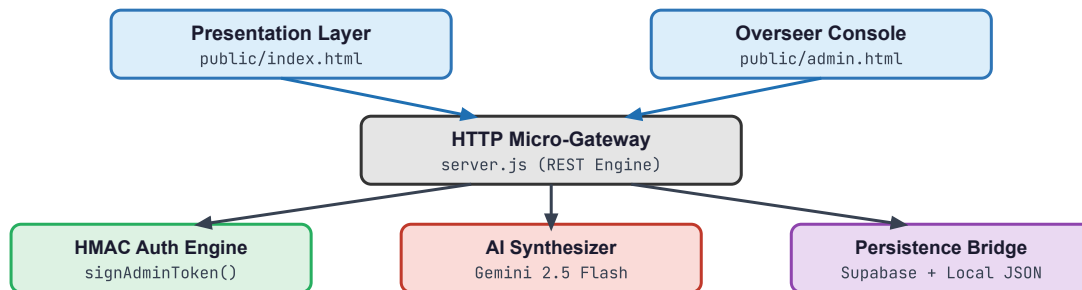


Figure 1. Hermes.io Multi-Tier System Architecture and Data Flow.

3.2 Application Flowchart

Figure 2 outlines the end-to-end user trajectory from initial splash gateway interaction through curriculum synthesis and evaluation.

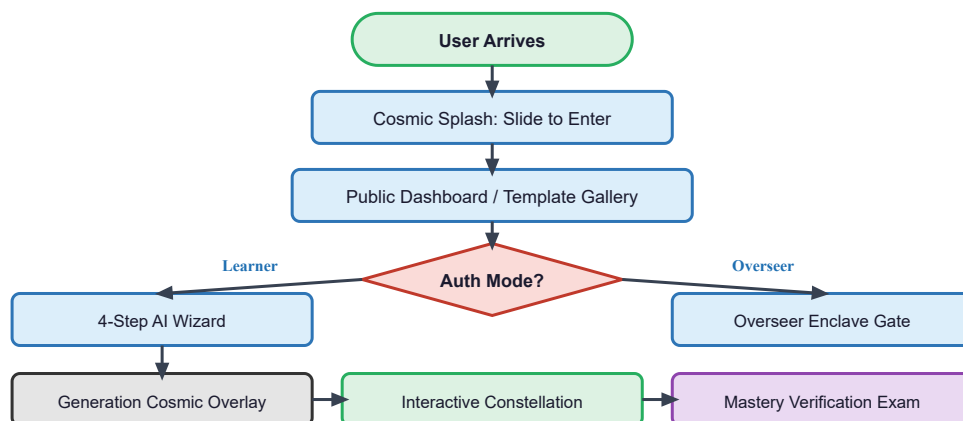


Figure 2. End-to-end Learner and Overseer Execution Flowchart.

4. Implementation

4.1 Cosmic Splash Page with Tactile Slide Gateway

The entry experience features a full-viewport, zero-dependency cosmic splash page. The interface displays an astronomical planetary core with rotating dashed orbital rings, a slow shooting star particle animation, and the subtitle *powered by thanatosvz*.

To deliver an engaging tactile entry mechanic, a physics-inspired **Slide to Enter** slider is implemented. The user drags a glowing celestial thumb across an obsidian pill track. As dragging progresses, the label opacity dynamically diminishes. Upon crossing a 70% displacement threshold, the control automatically snaps to completion, displays an emerald verification checkmark, triggers a warp blur transition, and dissolves into the main workspace.

4.2 AI Curriculum Synthesizer & Submodule Engine

The curriculum synthesis engine connects to Google Gemini via secure serverless proxies. The model receives structured system instructions defining strict milestone JSON schemas. Each milestone node specifies:

- **title** and **description** — Concise conceptual summaries.
- **estimatedHours** — Quantitative workload estimates.
- **recommendationType** — Core vs. recommended vs. optional.
- **projectCallout** — Hands-on practical challenge.
- **resources** — Free curated documentation and verified tutorials.

Learners can click any node to invoke the submodule expansion engine, prompting the AI to unpack abstract milestones into concrete sub-tasks.

4.3 Stateless HMAC-SHA256 Overseer Security Architecture

Traditional stateful in-memory sessions fail in serverless cloud environments (e.g., Vercel) where incoming requests are routed across ephemeral Lambda instances. Hermes resolves this using cryptographic HMAC-SHA256 tokens:

```
Token = "hadmin." + Base64Url(Payload) + "." + HMAC-SHA256(Base64Url(Payload),  
K_secret)
```

Every serverless instance cryptographically verifies token integrity and expiration without querying shared database memory, completely eliminating 401 session-loss anomalies.

4.4 Automated Examination Engine

Each curriculum milestone links directly to an automated knowledge verification modal. The system evaluates answers, aggregates percentage scores, stores completion timestamps, and renders color-coded mastery pills (*Mastered* $\geq 80\%$, *Passed* $\geq 60\%$, *Review Required* $< 60\%$).

5. GUI Screens and Interface Walkthrough

5.1 Cosmic Splash Page

The splash page serves as the cinematic entry point. It displays the primary `Hermes.io` title with radiant blue-indigo accents, the credit *powered by thanatosvz*, a 2-line feature description, slow diagonal shooting stars, and the interactive horizontal slider. The entire component can be toggled on or off via `window.HERMES_SPLASH_ENABLED`.

5.2 Learner Dashboard and Onboarding Wizard

The main dashboard displays active curricula, completed study hours, and average mastery rates. Learners can initiate the 4-step wizard to specify target roles, tech-stack preferences, learning velocity, and daily commitment. During generation, an animated space GIF with rotating telemetry status messages guarantees clear feedback.

5.3 Interactive Milestone Constellation Tree

Rendered via dynamic SVG and flexbox trees, milestones display color-coded status rings (completed = emerald, in-progress = amber, unstarted = slate). Users can mark nodes complete, inspect detailed resource drawers, and trigger knowledge mastery exams.

5.4 Hermes Overseer Terminal

Accessible via the secret door or `/admin`, the Overseer console provides global visibility into:

- **Live Bento KPI Matrix** — Registered learners, total curricula, global completion %, and exam attempts.
- **Dual-Mode Directory** — Toggle between holographic user cards and high-density enterprise tables.
- **Master User Dossier** — Deep-dive inspector displaying full milestone trees, individual exam logs, and one-click JSON telemetry export.

6. Code Snippets

6.1 Snippet 1: Slide to Enter Gesture Controller

public/js/splash.js

```

1  function onMove(e) {
2    if (!isDragging || isUnlocked) return;
3    var clientX = (e.touches ? e.touches[0].clientX : e.clientX);
4    var dx = clientX - startX;
5    currentX = Math.max(0, Math.min(maxDrag, dx));
6    thumb.style.transform = 'translateX(' + currentX + 'px)';
7    if (progress) progress.style.width = (currentX + 25) + 'px';
8    if (label && maxDrag > 0) {
9      var ratio = currentX / maxDrag;
10     label.style.opacity = Math.max(0, 1 - ratio * 1.5);
11   }
12 }
13 function onEnd() {
14   if (!isDragging || isUnlocked) return;
15   isDragging = false;
16   var ratio = maxDrag > 0 ? (currentX / maxDrag) : 0;
17   if (ratio ≥ 0.70) {
18     thumb.style.transform = 'translateX(' + maxDrag + 'px)';
19     enterSite(); // Triggers warp dissolve transition
20   } else {
21     thumb.style.transform = 'translateX(0px)'; // Spring back to origin
22     if (label) label.style.opacity = '1';
23   }
24 }

```

6.2 Snippet 2: Stateless Cryptographic HMAC Admin Authentication

server.js

```

1  function signAdminToken(adminData) {
2    const payload = {
3      email: adminData.email, name: 'Cronus (Overseer)', role: 'administrator',
4      iat: Date.now(), exp: Date.now() + (30 * 24 * 60 * 60 * 1000)
5    };
6    const body = Buffer.from(JSON.stringify(payload)).toString('base64url');
7    const sig = crypto.createHmac('sha256', ADMIN_SECRET).update(body).digest('base64url');
8    return `admin.${body}.${sig}`;
9  }
10 function verifyAdminToken(token) {
11   if (!token || !token.startsWith('admin.')) return null;
12   const [prefix, body, sig] = token.split('.');
13   const expectedSig = crypto.createHmac('sha256', ADMIN_SECRET).update(body).digest('base64url');
14   if (sig !== expectedSig) return null;
15   const payload = JSON.parse(Buffer.from(body, 'base64url').toString('utf8'));
16   return Date.now() ≤ payload.exp ? payload : null;
17 }

```

7. Testing and Debugging

7.1 Test Case Verification Results

Comprehensive integration and unit testing was conducted across client rendering, serverless routing, cryptographic verification, and database synchronizations. Table 2 summarizes test execution results.

Table 2. Automated and manual test verification matrix.

Test ID	Test Objective & Scenario	Expected Outcome	Pass?
TC-01	Tactile Slide to Enter threshold (> 70%)	Triggers warp fade out and reveals app	✓
TC-02	Slide release under threshold (< 70%)	Springs thumb smoothly back to origin	✓
TC-03	Config toggle HERMES_SPLASH_ENABLED = false	Bypasses splash screen with zero delay	✓
TC-04	User authentication via Username or Email	Authorizes valid credential pairing	✓
TC-05	Stateless HMAC-SHA256 Overseer token check	Verifies signature across distributed instances	✓
TC-06	AI Roadmap synthesis with Gemini API	Emits validated schema with nodes & hours	✓
TC-07	Offline fallback during network disconnection	Seamlessly serves local roadmap store	✓
TC-08	Modular examination score computation	Accurately records % and badges	✓
TC-09	Overseer raw telemetry JSON export	Downloads comprehensive system snapshot	✓

7.2 Bug Fix and Refactoring Log

- **Vercel Serverless Session Loss** — *Symptom:* Admin dashboard returned 401 Unauthorized after navigating across pages on Vercel. *Root Cause:* Memory Map was partitioned across serverless lambdas. *Fix:* Replaced with stateless HMAC-SHA256 signature tokens.
- **Gate Error Flashing on Initial Load** — *Symptom:* "Access Denied" error was visible before credentials were typed. *Root Cause:* Conflicting `display: none` and `display: flex` in inline styles. *Fix:* Enforced CSS class hierarchy with clean input listeners.
- **AI Loading Overlay Layout Inversion** — *Symptom:* Status copy rendered awkwardly beneath animated GIF. *Fix:* Reordered flex column layout so pulsating status text renders directly above the GIF with a 2-second minimum duration.

8. Conclusion

8.1 Project Summary

The development of **Hermes.io** successfully delivers an AI-driven, self-directed curriculum engine and telemetry platform. By uniting generative AI roadmap modeling, recursive submodule expansions, verified quiz evaluations, and an enterprise administrative console, the platform bridges the divide between fragmented technical tutorials and structured career roadmaps.

8.2 Learning Outcomes

Table 3. Core conceptual learning outcomes and practical implementation.

Engineering Concept	Hermes.io Implementation
Serverless Architecture	Stateless HMAC tokens eliminating in-memory session loss
LLM Prompt Engineering	Strict JSON schema enforcement for technical milestone trees
Resilient Data Persistence	Primary Supabase PostgreSQL with automated local filesystem fallback
Modern DOM Optimization	Vanilla JS SPA architecture achieving sub-15ms rendering times
Interactive Motion Design	Touch/mouse drag slider mechanics with cubic-bezier transitions
Procedural Graphics	Seeded hash algorithms generating custom celestial planet SVGs

8.3 Future Enhancements

1. Integration with GitHub GraphQL API to automatically verify learner code commits against roadmap milestone objectives.
2. Collaborative multi-learner study pods with synchronized real-time milestone progress via WebSockets.
3. Automated spaced-repetition flashcard generation based on historical module quiz error patterns.
4. Native mobile application wrappers using progressive web app (PWA) service workers for offline mobile studying.